Community Civic Issue Reporting System

CS23333

object oriented programming using java Project Report

Submitted by

HEMA A	**-**	**251001075**
IRISH S	**-**	**251001083**
LAKSHAYA P	**-**	**251001129**
LAVANYA K	**-**	**251001324**

Of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM
(An Autonomous Institution)

BONAFIDE CERTIFICATE

Certified that this project titled "**COMMUNITY CIVIC ISSUE REPORTING SYSTEM**" is the bonafide work of **HEMA(251001075), IRISH(251001083), LAKSHAYA(251001129), LAVANYA(251001324)** and who carried out the project work under my supervision.

SIGNATURE

Dr. P. Neelaveni
HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai – 602105

SIGNATURE

M. Janani
COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai – 602105

This project is submitted for **CS23333** object oriented programming using java held on _____

INTERNAL EXAMINAR

EXTERNAL EXAMINAR

Abstract:

The Community Civic Issue Reporting System is a Java-based desktop application developed to help citizens report and track various civic issues in their communities, such as road damage, water leakage, garbage accumulation, and drainage problems. Users can easily submit complaints through the system, while administrators can view, manage, and update the status of reported issues as Pending, In Progress, or Resolved. The application provides a centralized platform for complaint management, reduces manual work, and improves communication between citizens and administrators. It also allows users to monitor the progress of their complaints, promoting transparency and accountability. Overall, this system aims to make civic issue reporting easier, faster, and more organized while ensuring better management of community problems.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
1)	**INTRODUCTION** 1.1 Introduction 1.2 Purpose of the Project 1.3 Scope of the Project 1.4 Objectives of the Project	5-6
2)	**REQUIREMENTS AND FEASIBILITY** 2.1 Hardware Requirements 2.2 Software Requirements 2.3 Feasibility Study	7-8
3)	**SYSTEM ANALYSIS AND DESIGN** 3.1 Existing System 3.2 Proposed System 3.3 System Architecture Diagram 3.4 Use Case Diagram 3.5 Data Flow Diagram	9-12
4)	**DATABASE DESIGN** 4.1 Entity Relationship Diagram 4.2 Database Tables 4.3 Table Structure	13-15
5)	**IMPLEMENTATION** 5.1 Java OOP Concepts 5.2 Database Connectivity (JDBC) 5.3 CRUD Operations 5.4 Module Description	16-22
6)	**VERIFICATION AND TEST PLAN** 6.1 Testing Methods 6.2 Test Cases 6.3 Test Results	23-29
7)	**SECURITY, LIMITATIONS AND MAINTENANCE** 7.1 Security Measures 7.2 Limitations 7.3 Maintenance	30
8)	**CONCLUSION AND FUTURE WORK** 8.1 Conclusion 8.2 Future Enhancements	31
	REFERENCES	32
	APPENDICES A. SQL Schema B. Output Screenshots	33-39

1. INTRODUCTION:

1.1 INTRODUCTION:

The Community Civic Issue Reporting System is an innovative software solution designed to help citizens report and manage various civic issues in their communities. It provides a centralized platform for reporting problems such as road damage, water leakage, garbage accumulation, drainage issues, and other public infrastructure-related concerns.

This system introduces automation into the complaint management process, allowing citizens to submit complaints easily while administrators can view, manage, and update their status. By integrating technology into civic administration, the system reduces manual work, improves communication, and provides a more efficient and user-friendly experience. It also helps ensure transparency and accountability by allowing users to track the progress of their complaints, making civic issue management simpler and more organized.

1.2 PURPOSE OF THIS PROJECT:

The purpose of this project is to develop a user-friendly Community Civic Issue Reporting System named CivicTrack to help citizens report and track civic problems efficiently. The system aims to:

- Allow citizens to report various civic issues easily.
- Enable users to track the status of their submitted complaints.
- Help administrators manage and monitor reported issues.
- Display the total number of complaints and their statuses, such as Pending, In Progress, and Resolved.
- Provide an organized platform for handling and resolving community problems.
- Reduce manual complaint management and improve communication between citizens and administrators.
- Promote transparency, accountability, and faster resolution of civic issues.

1.3 SCOPE OF THE PROJECT:

The Community Civic Issue Reporting System, named CivicTrack, aims to provide an easy and efficient platform for citizens to report and track civic issues. Developed using Java and JDBC with a MySQL database, the system ensures efficient complaint management and reduces manual work. It allows citizens to report problems such as road damage, water leakage, garbage accumulation, and drainage issues. Administrators can view complaints, monitor their progress, and update their status as Pending, In Progress, or Resolved. The system helps store and manage complaint details, track issue status, improve communication between citizens and administrators, and promote transparency and accountability in resolving community problems.

1.4 OBJECTIVES OF THE PROJECT:

The main objective of the Community Civic Issue Reporting System (CivicTrack) is to provide an efficient platform for reporting, tracking, and managing civic issues. The system aims to:

- Allow citizens to submit complaints about various civic problems.
- Provide an easy-to-use interface for reporting issues.
- Enable users to track complaint status.
- Help administrators monitor and manage reported complaints.
- Update complaint statuses as Pending, In Progress, or Resolved.
- Maintain complaint records in a centralized database.
- Reduce manual work and improve complaint management.
- Ensure transparency and accountability in resolving community issues.

2. REQUIREMENTS AND FEASIBILITY:

2.1 HARDWARE REQUIREMENTS:

The hardware requirements for developing and running the Community Civic Issue Reporting System (CivicTrack) are:

- Processor: Intel Core i3 or higher.
- RAM: Minimum 4 GB.
- Hard Disk: Minimum 250 GB.
- Monitor: Standard LED Monitor.
- Keyboard: Standard Keyboard.
- Mouse: Standard Mouse.
- Internet Connection: Not required for basic local operation; required only for online features, if implemented.
- System: Desktop or Laptop.

2.2 SOFTWARE REQUIREMENTS:

The software requirements for developing and running the Community Civic Issue Reporting System (CivicTrack) are:

- Operating System: Windows 10 or above.
- Programming Language: Java.
- Database: MySQL.
- Database Connectivity: JDBC.
- IDE: NetBeans / Eclipse / IntelliJ IDEA.
- Frontend: Java Swing.
- Backend: Java.
- Web Browser: Google Chrome or Microsoft Edge.
- Server: Apache Tomcat (if required).

2.3 FEASIBILITY STUDY:

The feasibility study is conducted to determine whether the Community Civic Issue Reporting System (CivicTrack) is practical and suitable for implementation. It includes the following:

1. Technical Feasibility:

The system is developed using Java, JDBC, and MySQL, which provide reliable tools for managing civic complaints. The required hardware and software are easily available.

2. Economic Feasibility:

The project is cost-effective because it uses commonly available software and tools. It reduces paperwork and manual efforts, minimizing administrative costs.

3. Operational Feasibility:

The system is simple and user-friendly. Citizens can easily submit complaints, while administrators can manage complaints and update their statuses.

4. Schedule Feasibility:

The project can be developed and implemented within the given time using the available resources and technologies.

5. Maintenance Feasibility:

The system can be maintained and updated easily. New features can be added in the future to improve its functionality and performance.

3. SYSTEM ANALYSIS AND DESIGN:

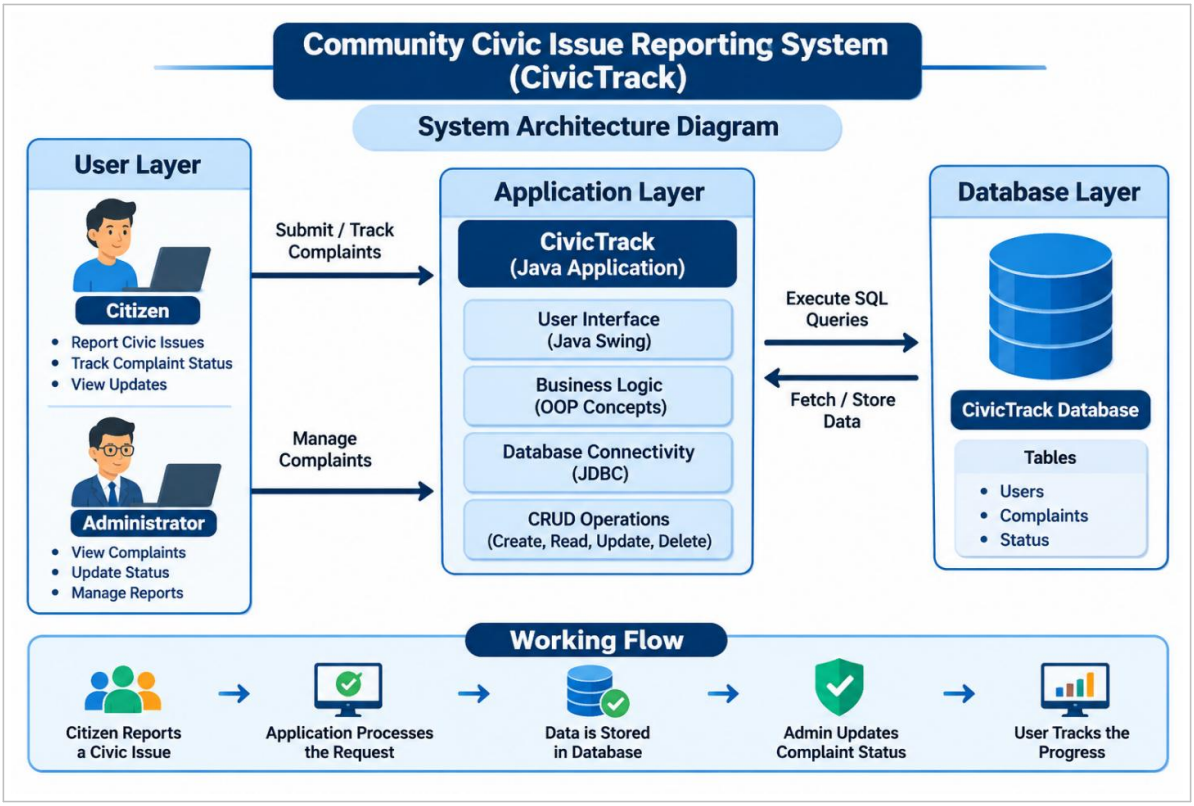
3.1 EXISTING SYSTEM:

In the existing system, civic issues such as road damage, water leakage, garbage accumulation, and drainage problems are generally reported through manual methods, phone calls, or direct complaints to local authorities. These methods require more time and effort, and it is difficult for citizens to track the progress of their complaints. Administrators may face difficulties in maintaining complaint records and monitoring the status of each issue. The lack of a centralized platform can lead to delays, poor communication, and inefficient complaint management. Therefore, a computerized system is needed to make the reporting and management process easier and more effective.

3.2 PROPOSED SYSTEM:

The proposed Community Civic Issue Reporting System (CivicTrack) is designed to provide an efficient and user-friendly platform for reporting and managing civic issues. Citizens can submit complaints related to road damage, water leakage, garbage accumulation, and drainage problems through the system. Users can also track their complaints and check their current status. The administrator can log in, view all submitted complaints, monitor their progress, and update their status as Pending, In Progress, or Resolved. The system maintains complaint records in a centralized database, reduces manual work, improves communication, and ensures better transparency and efficiency in civic issue management.

3.3 SYSTEM ARCHITECTURE DIAGRAM:

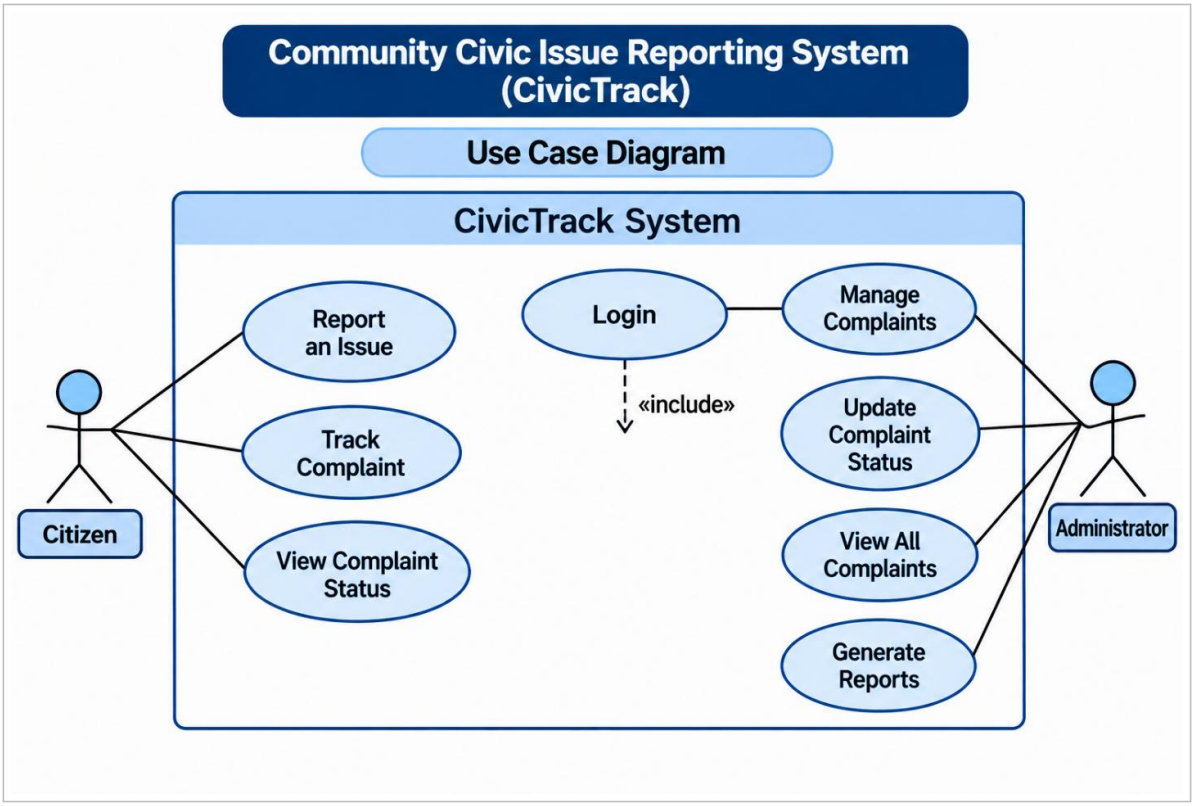


The CivicTrack system follows a layered architecture consisting of Java Swing, Java application logic, DAO, JDBC, and MySQL database. Citizens can submit and track complaints, while administrators can view complaints and update their status. JDBC connects the Java application to the MySQL database for storing and retrieving complaint details.

Main Components:

- User Interface: Java Swing
- Application Logic: Java
- DAO Layer: ComplaintDAO
- Connectivity: JDBC
- Database: MySQL (admins and complaints tables)

3.4 USE CASE DIAGRAM:

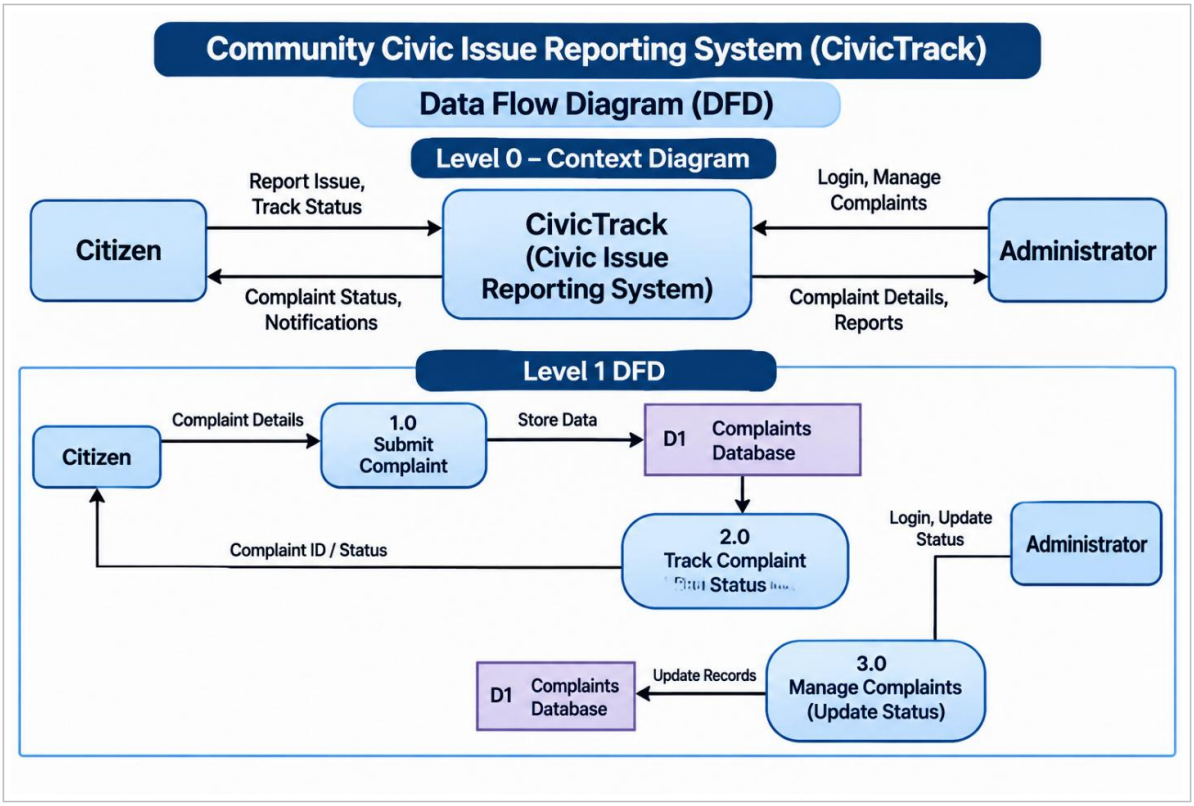


The Use Case Diagram represents the interactions between citizens and administrators in the CivicTrack system. Citizens can submit civic complaints and track their status, while administrators can log in, view complaints, manage them, and update their status.

Main Actors:

- Citizen: Reports issues and tracks complaints.
- Administrator: Logs in, views and manages complaints, and updates complaint status.

3.5 DATA FLOW DIAGRAM:



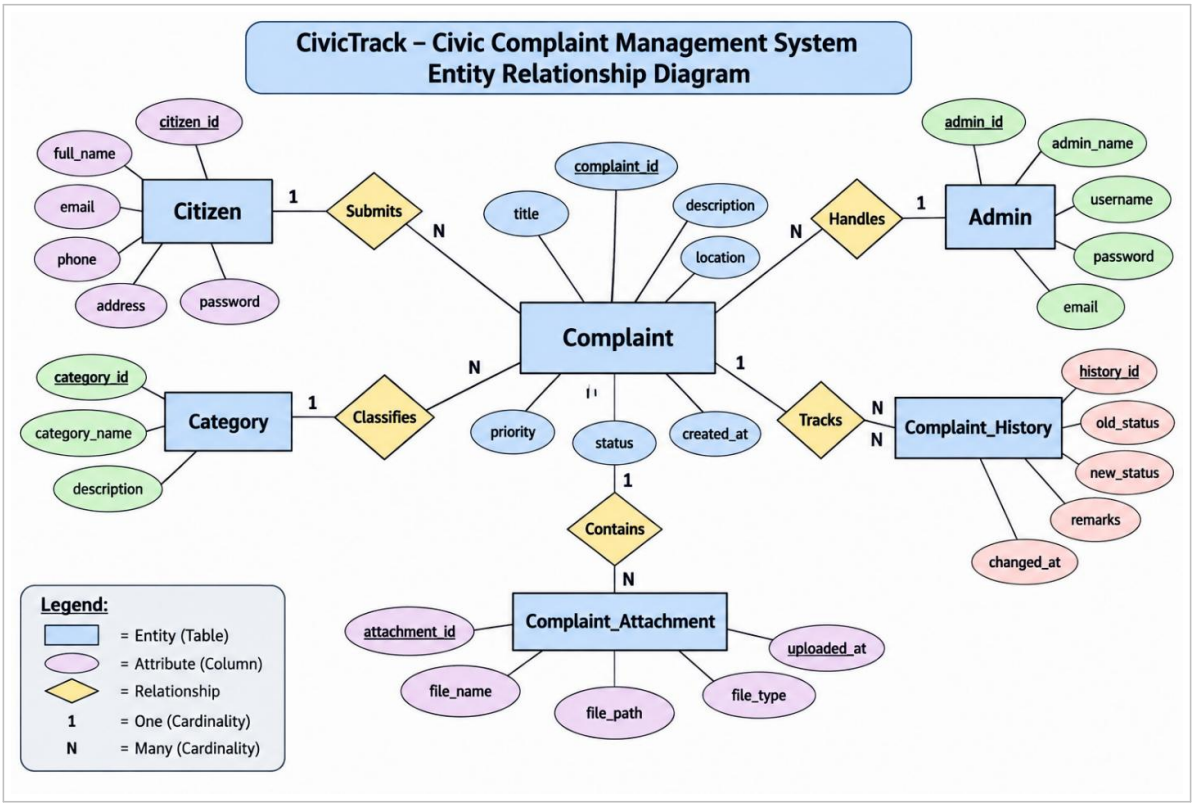
The Data Flow Diagram represents how information flows within the CivicTrack system. Citizens submit civic complaints, which are stored in the MySQL database. Administrators view complaints and update their status. The system retrieves and displays the updated complaint information to users.

Main Processes:

- Complaint Submission: Citizens submit civic issues.
- Complaint Tracking: Users check complaint status.
- Complaint Management: Admins view and manage complaints.
- Database: Stores admin details and complaint records.

4. DATABASE DESIGN:

4.1 ENTITY RELATIONSHIP DIAGRAM:



The Entity Relationship Diagram represents the database structure of CivicTrack and shows the relationship between its entities. The system contains two main entities: Admins and Complaints. The Admins table stores administrator login details, while the Complaints table stores civic issue information, including priority and status.

Main Entities:

- Admins: Stores admin ID, username, and password.
- Complaints: Stores complaint ID, title, category, description, location, reporter details, priority, and status.
- Relationship: Administrators manage complaint records by viewing and updating their status. However, the current database schema does not establish a direct foreign key relationship between the Admins and Complaints tables. The relationship is maintained through application-level operations.

4.2 DATABASE TABLE:

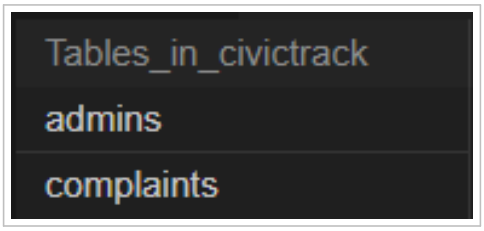
The CivicTrack database consists of two main tables: Admins and Complaints. The Admins table stores administrator login details, while the Complaints table stores civic issue details, reporter information, priority levels, and complaint status. These tables help maintain and manage complaint records efficiently.

Database Tables:

1. Admins
2. Complaints

SQL Query:

```
USE civictrack;
SHOW TABLES;
```



4.3 TABLE STRUCTURE:

The table structure describes the columns, data types, and constraints used in the CivicTrack database. It consists of two tables: Admins and Complaints. The Admins table stores administrator login details, while the Complaints table maintains civic issue information, reporter details, priority, and complaint status.

Main Tables:

1. Admins Table: Stores admin ID, username, and password.
2. Complaints Table: Stores complaint ID, title, category, description, location, reporter details, priority score, priority level, status, upvotes, admin remarks, and timestamps.

SQL Queries:

```
DESCRIBE admins;
```

Field	Type	Null	Key	Default	Extra
admin_id	int	NO	PRI	NULL	auto_increment
username	varchar(50)	NO	UNI	NULL	
password	varchar(100)	NO		NULL	

```
DESCRIBE complaints;
```

Field	Type	Null	Key	Default	Extra
complaint_id	int	NO	PRI	NULL	auto_increment
title	varchar(150)	NO		NULL	
category	varchar(50)	NO		NULL	
description	text	NO		NULL	
location	varchar(200)	NO		NULL	
reporter_name	varchar(100)	NO		NULL	
reporter_contact	varchar(100)	NO		NULL	
priority_score	int	NO	MUL	0	
priority_level	varchar(20)	NO		Low	
status	enum('Pending','In Progress','Resolved')	NO	MUL	Pending	
upvotes	int	NO		0	
admin_remarks	varchar(255)	YES		NULL	
created_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
updated_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT_TIMESTAMP

5. IMPLEMENTATION:

5.1 JAVA OOP CONCEPT:

- **Encapsulation:** Protects data by using private variables and getter/setter methods.
- **Inheritance:** Allows one class to acquire properties and methods from another class.
- **Polymorphism:** Allows methods to perform different operations through overloading or overriding.
- **Abstraction:** Hides internal implementation details and displays essential functionalities.

1. Encapsulation:

CivicTrack uses encapsulation in the Complaint class. All complaint attributes are declared private and accessed through public getter and setter methods. This protects data and provides controlled access to complaint information.

```
package model;

import java.sql.Timestamp;

public class Complaint {
    private int id;
    private String title;
    private String category;
    private String description;
    private String location;
    private String reporterName;
    private String reporterContact;
    private int priorityScore;
    private String priorityLevel;
    private String status;
    private int upvotes;
    private String adminRemarks;
    private Timestamp createdAt;

    public Complaint() {}

    public Complaint(String title, String category, String description, String location,
                     String reporterName, String reporterContact) {
        this.title = title;
        this.category = category;
        this.description = description;
        this.location = location;
        this.reporterName = reporterName;
        this.reporterContact = reporterContact;
        this.status = "Pending";
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getTitle() { return title; }
```

```
    public void setTitle(String title) { this.title = title; }

    public String getCategory() { return category; }
    public void setCategory(String category) { this.category = category; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public String getLocation() { return location; }
    public void setLocation(String location) { this.location = location; }

    public String getReporterName() { return reporterName; }
    public void setReporterName(String reporterName) { this.reporterName =
        reporterName; }

    public String getReporterContact() { return reporterContact; }
    public void setReporterContact(String reporterContact) { this.reporterContact =
        reporterContact; }

    public int getPriorityScore() { return priorityScore; }
    public void setPriorityScore(int priorityScore) { this.priorityScore =
        priorityScore; }

    public String getPriorityLevel() { return priorityLevel; }
    public void setPriorityLevel(String priorityLevel) { this.priorityLevel =
        priorityLevel; }

    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }

    public int getUpvotes() { return upvotes; }
    public void setUpvotes(int upvotes) { this.upvotes = upvotes; }

    public String getAdminRemarks() { return adminRemarks; }
    public void setAdminRemarks(String adminRemarks) { this.adminRemarks =
        adminRemarks; }

    public Timestamp getCreatedAt() { return createdAt; }
    public void setCreatedAt(Timestamp createdAt) { this.createdAt = createdAt; }
}
```

2. Inheritance

Inheritance is an important OOP concept that allows one class to acquire the properties and methods of another class.

In the CivicTrack project, inheritance is implemented in the AdminDashboardFrame class, which extends the JFrame class provided by Java Swing.

Code Implementation:

```
public class AdminDashboardFrame extends JFrame {

    private final ComplaintDAO dao = new ComplaintDAO();

    // Admin dashboard implementation
}
```

Explanation:

The AdminDashboardFrame class inherits the features and methods of the JFrame class. This allows the application to create and manage the administrator dashboard window using existing Swing functionality without implementing window management from scratch.

3. Polymorphism

Polymorphism is an important Object-Oriented Programming concept that allows a method to have different implementations.

In the CivicTrack project, polymorphism is implemented through method overriding in the AdminDashboardFrame class.

Code Implementation:

```
model = new DefaultTableModel(cols, 0) {
    @Override
    public boolean isCellEditable(int r, int c) {
        return false;
    }
};
```

Explanation:

The isCellEditable() method of the DefaultTableModel class is overridden to return false. This prevents users from directly editing the complaint details displayed in the admin dashboard table. It demonstrates runtime polymorphism through method overriding.

4. Abstraction

Abstraction is an Object-Oriented Programming concept that hides internal implementation details and exposes only the essential functionality to the user.

In the CivicTrack project, abstraction is implemented using the Data Access Object (DAO) class. The ComplaintDAO class handles database operations such as inserting, retrieving, updating, and deleting complaints.

Code Implementation:

```
public class ComplaintDAO {

    public int addComplaint(Complaint c) throws SQLException {
        // Database insertion logic
    }

    public Complaint getComplaintById(int id) throws SQLException {
        // Retrieve complaint from database
    }

    public boolean updateStatus(int id, String status, String remarks)
            throws SQLException {
        // Update complaint status
    }

    public boolean deleteComplaint(int id) throws SQLException {
        // Delete complaint from database
    }
}
```

Explanation:

The ComplaintDAO class hides the internal database operations, including SQL queries and connection handling. Other classes can access database functionality by calling its public methods without needing to understand the underlying implementation.

This provides abstraction and separates database operations from the user interface.

5.2 DATABASE CONNECTIVITY:

CivicTrack uses JDBC (Java Database Connectivity) to connect the Java application with the MySQL database. The DBConnection class establishes the database connection, allowing the application to store, retrieve and update complaint information.

Main Functions:

- Establishes a connection with MySQL.
- Uses JDBC to execute database operations.
- Retrieves and stores complaint records.
- Supports communication between Java and MySQL.

```
package database;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL =
        "jdbc:mysql://localhost:3306/civictrack?useSSL=false&serverTimezone=UTC&allowPub
            licKeyRetrieval=true";
```

```
    private static final String USER = "root";
    private static final String PASSWORD = "root";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Note: *The username and password specified in the DBConnection class are MySQL database credentials used by the application to establish a database connection. They are separate from the admin login credentials stored in the admins table. For this academic project, default local database credentials are used for testing purposes. In a real-world deployment, credentials should be stored securely and should not be hardcoded in the source code.*

5.3 CRUD OPERATIONS:

CivicTrack uses CRUD operations to manage civic complaint records in the MySQL database through JDBC.

Main Operations:

1. **Create:** Adds new complaints to the database.
2. **Read:** Retrieves and displays complaint details.
3. **Update:** Updates complaint status, priority and admin remarks.
4. **Delete:** Removes complaint records from the database.

Main File: ComplaintDAO.java

Database Table: complaints

```
package dao;

import database.DBConnection;
import model.Complaint;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ComplaintDAO {

    public int addComplaint(Complaint c) throws SQLException {
        String sql = "INSERT INTO complaints (title, category, description, location,
            reporter_name, reporter_contact) VALUES (?,?,?,?,?,?)";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql,
                 Statement.RETURN_GENERATED_KEYS)) {

            ps.setString(1, c.getTitle());
            ps.setString(2, c.getCategory());
            ps.setString(3, c.getDescription());
            ps.setString(4, c.getLocation());
            ps.setString(5, c.getReporterName());
            ps.setString(6, c.getReporterContact());
```

```
            ps.executeUpdate();

            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) {
                    return rs.getInt(1);
                }
            }
        }
        return -1;
    }

    public Complaint getComplaintById(int id) throws SQLException {
        String sql = "SELECT * FROM complaints WHERE complaint_id = ?";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);

            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    Complaint c = new Complaint();
                    c.setId(rs.getInt("complaint_id"));
                    c.setTitle(rs.getString("title"));
                    c.setCategory(rs.getString("category"));
                    c.setDescription(rs.getString("description"));
                    c.setLocation(rs.getString("location"));
                    c.setReporterName(rs.getString("reporter_name"));
                    c.setReporterContact(rs.getString("reporter_contact"));
                    c.setStatus(rs.getString("status"));
                    return c;
                }
            }
        }
        return null;
    }

    public boolean updateStatus(int id, String status, String remarks) throws
        SQLException {
        String sql = "UPDATE complaints SET status = ?, admin_remarks = ? WHERE
            complaint_id = ?";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, status);
            ps.setString(2, remarks);
            ps.setInt(3, id);

            return ps.executeUpdate() > 0;
        }
    }

    public boolean deleteComplaint(int id) throws SQLException {
        String sql = "DELETE FROM complaints WHERE complaint_id = ?";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, id);
```

```
            return ps.executeUpdate() > 0;
        }
    }
}
```

5.4 MODULE DESCRIPTION:

CivicTrack consists of the following modules, each performing a specific function in the complaint management system.

1. **User Module:** Allows citizens to submit civic complaints and track their status.
2. **Admin Module:** Allows administrators to log in, view complaints and manage complaint records.
3. **Complaint Management Module:** Handles complaint submission, retrieval, updating and deletion.
4. **Priority Management Module:** Manages complaint priority scores and priority levels based on the priority calculation implemented in the application. The calculated priority information is stored in the database to help administrators identify important complaints.
5. **Database Module:** Uses JDBC and MySQL to store and retrieve administrator and complaint information.
6. **Complaint Tracking Module:** Allows citizens to check complaint status and view updates.

6. VERIFICATION AND TEST PLAN:

6.1 TESTING METHODS:

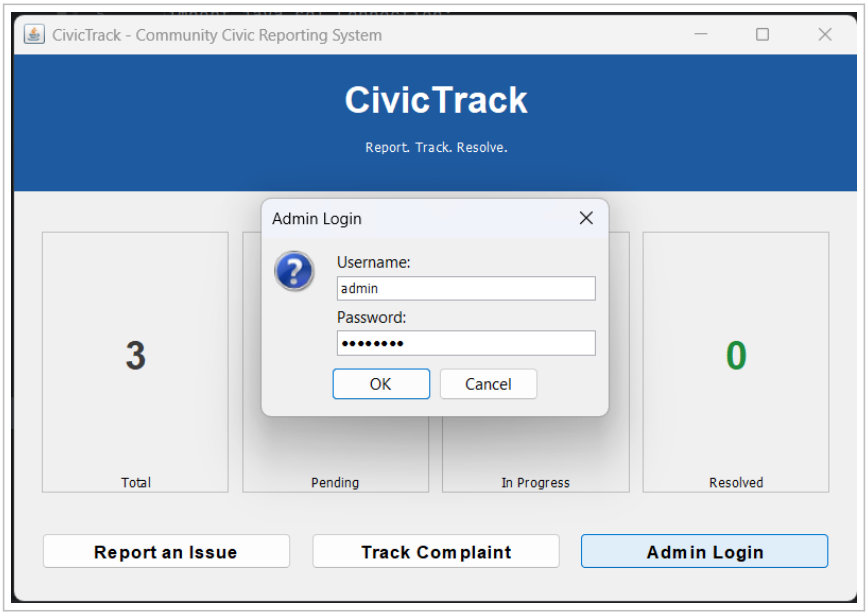
CivicTrack uses various testing methods to ensure that the application functions correctly and meets its requirements.

1. **Unit Testing:** Tests individual functions such as complaint submission and priority calculation.
2. **Integration Testing:** Checks the connection between Java, JDBC and MySQL.
3. **Functional Testing:** Verifies admin login, complaint submission, complaint tracking and status updates.
4. **System Testing:** Checks the complete application to ensure all modules work properly together.

6.2 TEST CASES:

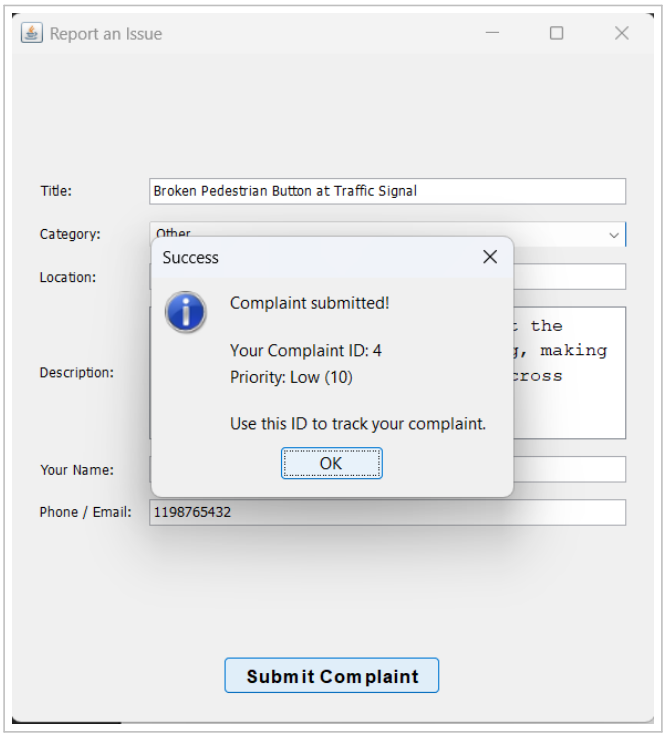
Test Result 1: Admin Login

The administrator login was tested using valid credentials. The system successfully authenticated the administrator and opened the dashboard.



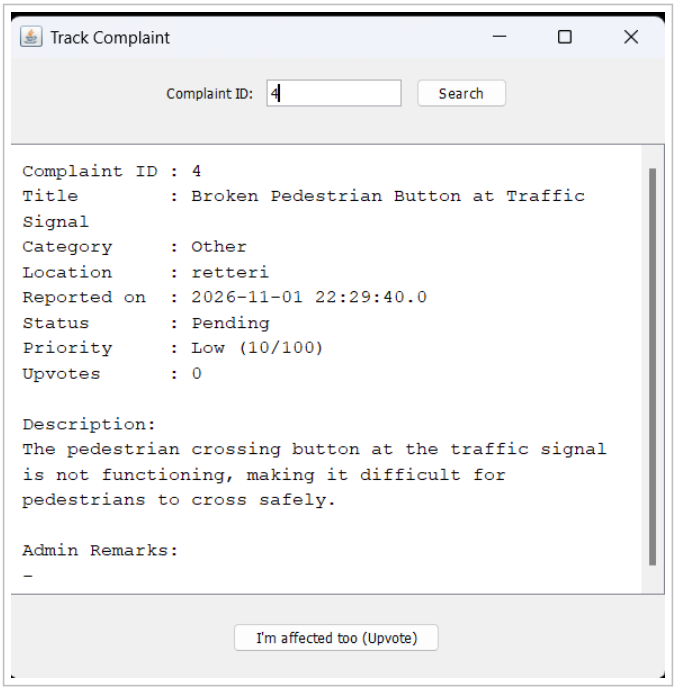
Test Result 2: Complaint Submission

A new civic complaint was submitted through the application. The complaint details were successfully saved in the MySQL database.



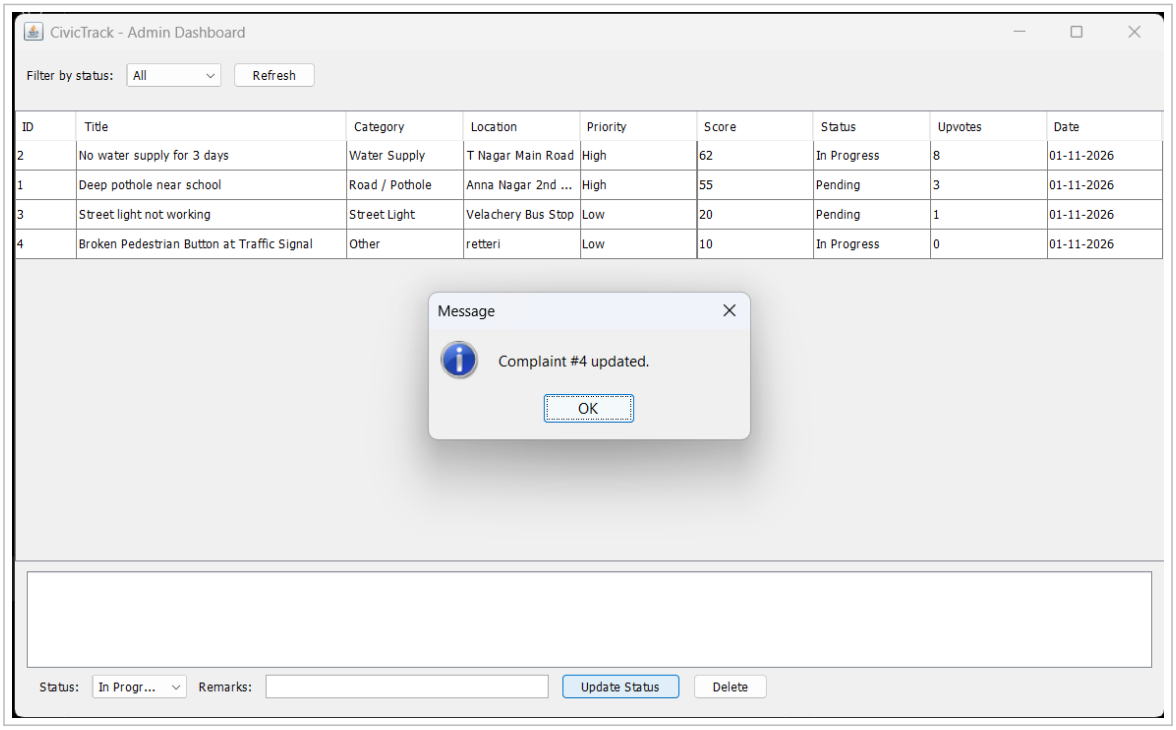
Test Result 3: Complaint Tracking

The complaint tracking feature was tested using a valid complaint ID. The system successfully displayed the complaint details and its current status.



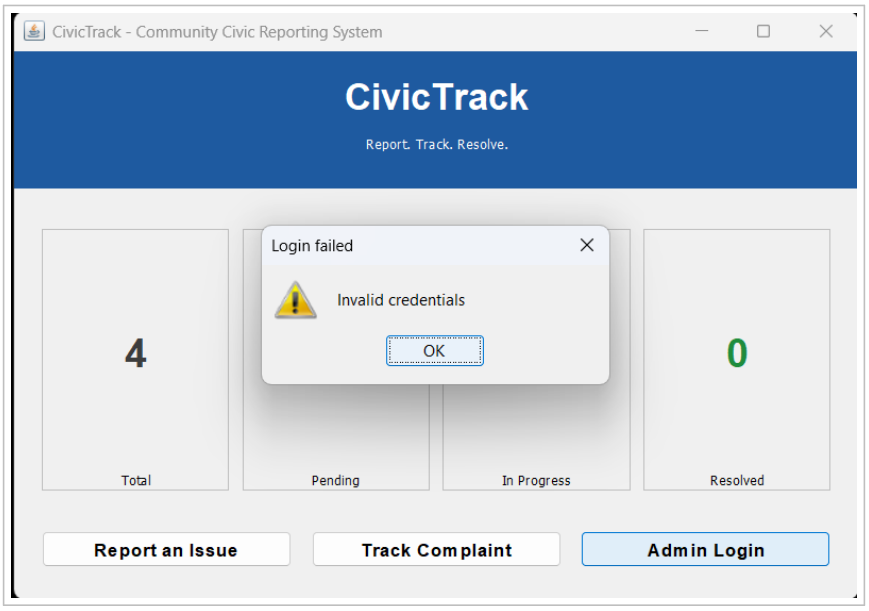
Test Result 4: Status Update

The administrator updated the complaint status through the dashboard. The updated status and remarks were successfully saved in the database.



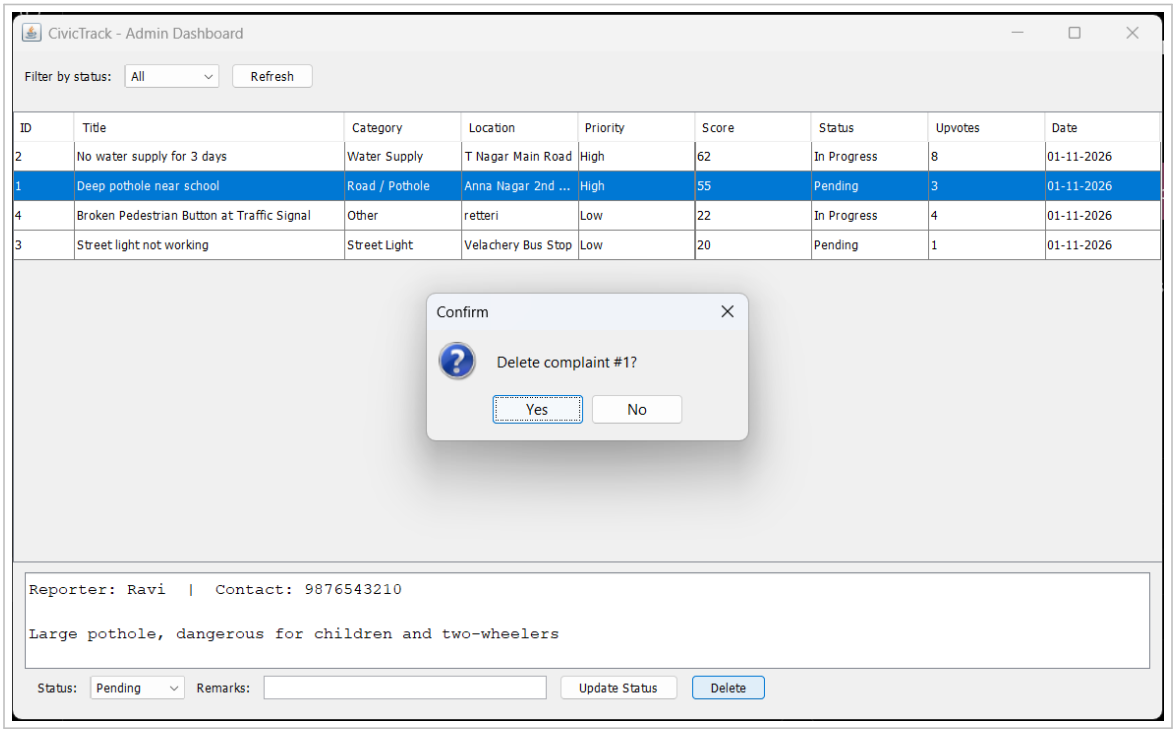
Test Result 5: Invalid Admin Login

The login functionality was tested using an incorrect administrator password. The system rejected the login attempt and displayed an error message.



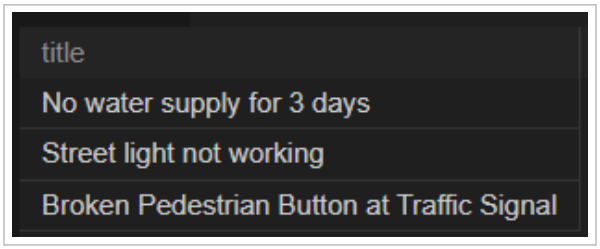
Test Result 6: Complaint Deletion

The complaint deletion functionality was tested by deleting a reported issue. The selected complaint was successfully removed from the application and database.



Test Result 7: Database Connectivity and Storage

Database connectivity was verified by submitting a new complaint and checking the records in the MySQL database. The submitted complaint appeared in the database, and the deleted complaint was no longer present.



6.3 TEST RESULTS:

The CivicTrack application was tested to verify its major functionalities, including administrator login, complaint submission, complaint tracking, status updates, invalid login attempts, complaint deletion and database connectivity.

The testing results confirmed that all seven test cases produced the expected outputs. The application successfully performed complaint management operations and maintained proper communication with the MySQL database.

Test results are presented in Section 6.2 through the corresponding screenshots.

Conclusion:

The testing process confirms that CivicTrack functions as expected and meets its basic requirements for civic complaint management.

7. SECURITY, LIMITATIONS AND MAINTENANCE:

7.1 SECURITY MEASURES

CivicTrack implements basic security measures to protect application data and administrator access.

1. **Admin Authentication:** Verifies administrator login credentials before allowing access to the admin dashboard.
2. **Data Protection:** Uses private variables and getter/setter methods to control access to complaint data.
3. **Prepared Statements:** Uses JDBC PreparedStatement to execute SQL queries and reduce the risk of SQL injection.
4. **Access Control:** Restricts complaint management operations to administrators.
5. **Password Storage:** The current application stores administrator passwords in plain text. Password hashing has not been implemented in the current version and is considered a future security improvement.

7.2 LIMITATIONS:

The CivicTrack application has some limitations:

1. Requires an active database connection to function.
2. Currently supports only administrator login.
3. Does not provide real-time notifications to citizens.
4. Requires manual updates from administrators.
5. Depends on a local MySQL database.

7.3 MAINTENANCE:

Regular maintenance helps CivicTrack function smoothly and reliably.

1. **Database Maintenance:** Regularly checks and backs up complaint records.
2. **Bug Fixing:** Identifies and fixes errors in the application.
3. **Performance Monitoring:** Ensures smooth application performance.
4. **Security Updates:** Updates login and database security measures.
5. **System Updates:** Improves existing features and adds new functionalities when required.

8. CONCLUSION AND FUTURE WORK:

8.1 CONCLUSION:

CivicTrack is a Java-based civic complaint management system developed to simplify the process of reporting and tracking public issues. The application allows citizens to submit complaints and track their status, while administrators can view, manage and update complaints. Using Java Swing, JDBC and MySQL, the system provides efficient complaint management, priority calculation and data storage. The project successfully demonstrates the use of Java OOP concepts and database connectivity.

8.2 FUTURE ENHANCEMENTS:

The following features can be added to improve CivicTrack in the future:

1. **Mobile Application:** Develop a mobile version for easier access.
2. **Email Notifications:** Send automatic updates about complaint status.
3. **GPS Location:** Allow citizens to share the exact location of civic issues.
4. **Image Upload:** Allow users to attach photos of reported issues.
5. **Online Deployment:** Make the application accessible through the internet.
6. **Advanced Analytics:** Generate reports and statistics about complaints.

REFERENCES

1. Oracle Java Documentation – https://docs.oracle.com/en/java/
2. MySQL Documentation – https://dev.mysql.com/doc/
3. Java JDBC Documentation – https://docs.oracle.com/javase/tutorial/jdbc/
4. Java Swing Documentation – https://docs.oracle.com/javase/tutorial/uiswing/
5. GeeksforGeeks – https://www.geeksforgeeks.org/
6. W3Schools Java Tutorial – https://www.w3schools.com/java/

APPENDIX A

SQL SCHEMA:

A.1 Database Creation

```
DROP DATABASE IF EXISTS civictrack;
CREATE DATABASE civictrack;
USE civictrack;
```

A.2 Admin Table Creation

```
CREATE TABLE admins (
    admin_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(100) NOT NULL
);
```

A.3 Complaints Table Creation

```
CREATE TABLE complaints (
    complaint_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    category VARCHAR(50) NOT NULL,
    description TEXT NOT NULL,
    location VARCHAR(200) NOT NULL,
    reporter_name VARCHAR(100) NOT NULL,
    reporter_contact VARCHAR(100) NOT NULL,
    priority_score INT NOT NULL DEFAULT 0,
    priority_level VARCHAR(20) NOT NULL DEFAULT 'Low',
    status ENUM('Pending','In Progress','Resolved') NOT NULL DEFAULT 'Pending',
    upvotes INT NOT NULL DEFAULT 0,
    admin_remarks VARCHAR(255) DEFAULT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

A.4 Index Creation

```
CREATE INDEX idx_status ON complaints(status);
CREATE INDEX idx_priority ON complaints(priority_score);
```

A.5 Admin Sample Data

```
INSERT INTO admins (username, password)
VALUES ('admin', 'admin123');
```

A.6 Complaint Sample Data

```
INSERT INTO complaints
(title, category, description, location, reporter_name, reporter_contact,
    priority_score, priority_level, status, upvotes)
VALUES
('Broken Pedestrian Button at Traffic Signal', 'Other', 'Pedestrian button at traffic
    signal is not working', 'Retteri', 'Not Provided', 'Not Provided', 10, 'Low', 'In
    Progress', 0);
```

A.7 SQL Queries

1. Display complaint status count

```
SELECT status, COUNT(*)
FROM complaints
GROUP BY status;
```

2. Display complaints according to priority

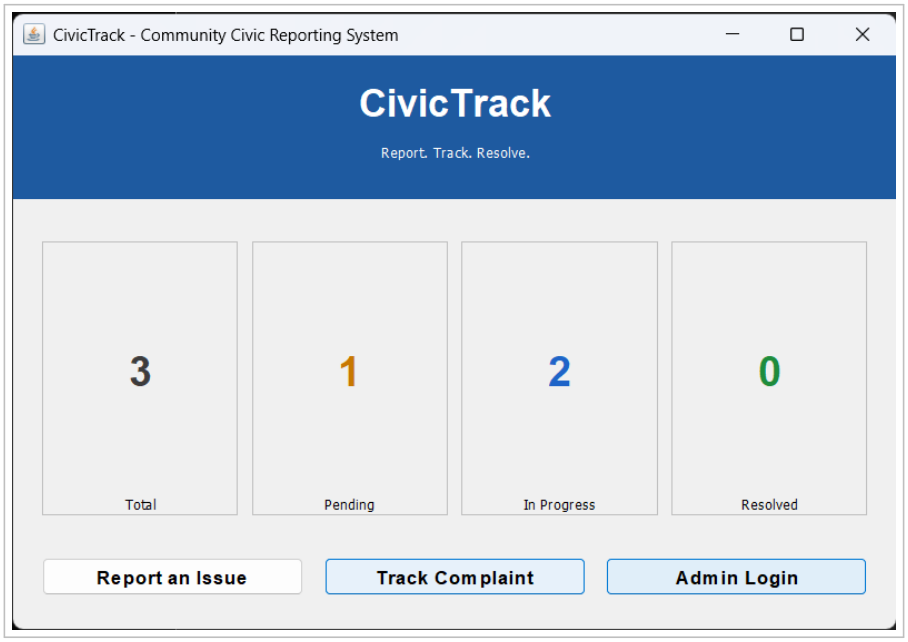
```
SELECT *
FROM complaints
ORDER BY priority_score DESC;
```

3. Display total complaints in each category

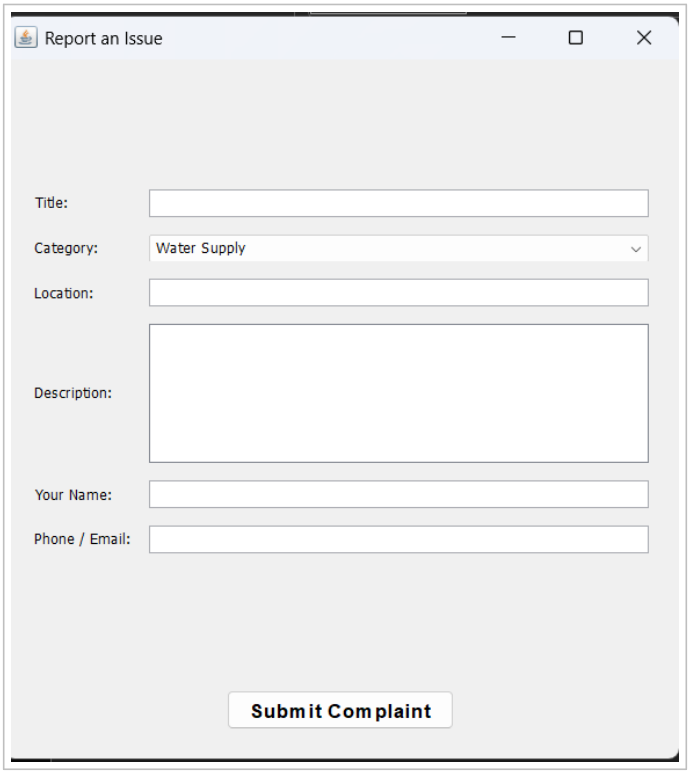
```
SELECT category, COUNT(*) AS total
FROM complaints
GROUP BY category;
```

OUTPUT SCREENSHOTS:

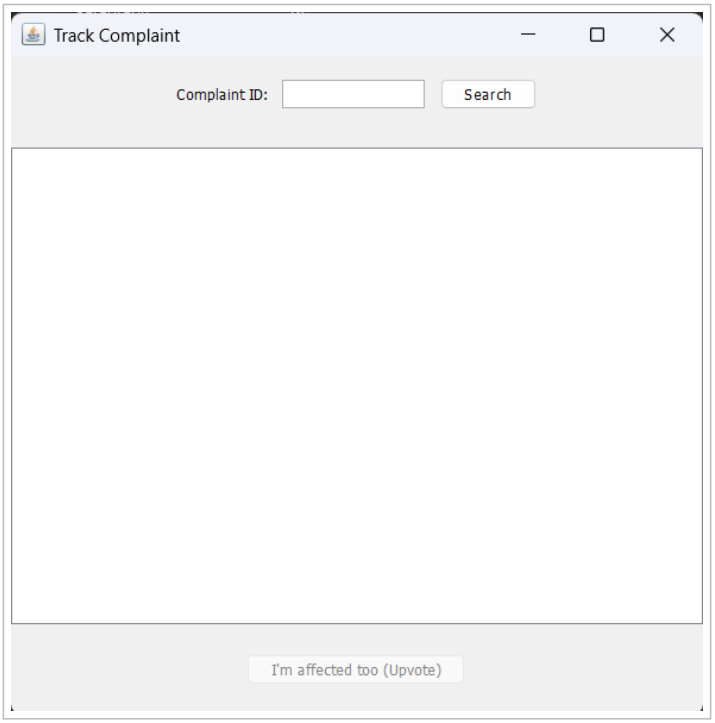
B.1 Home Page



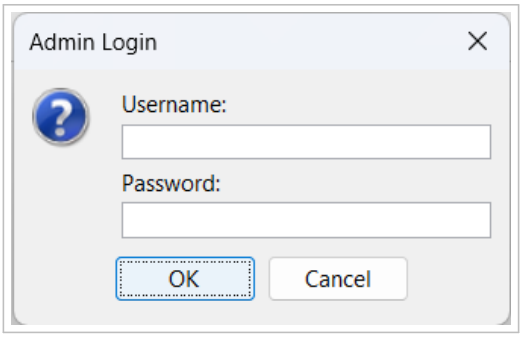
B.2 Complaint Submission Page



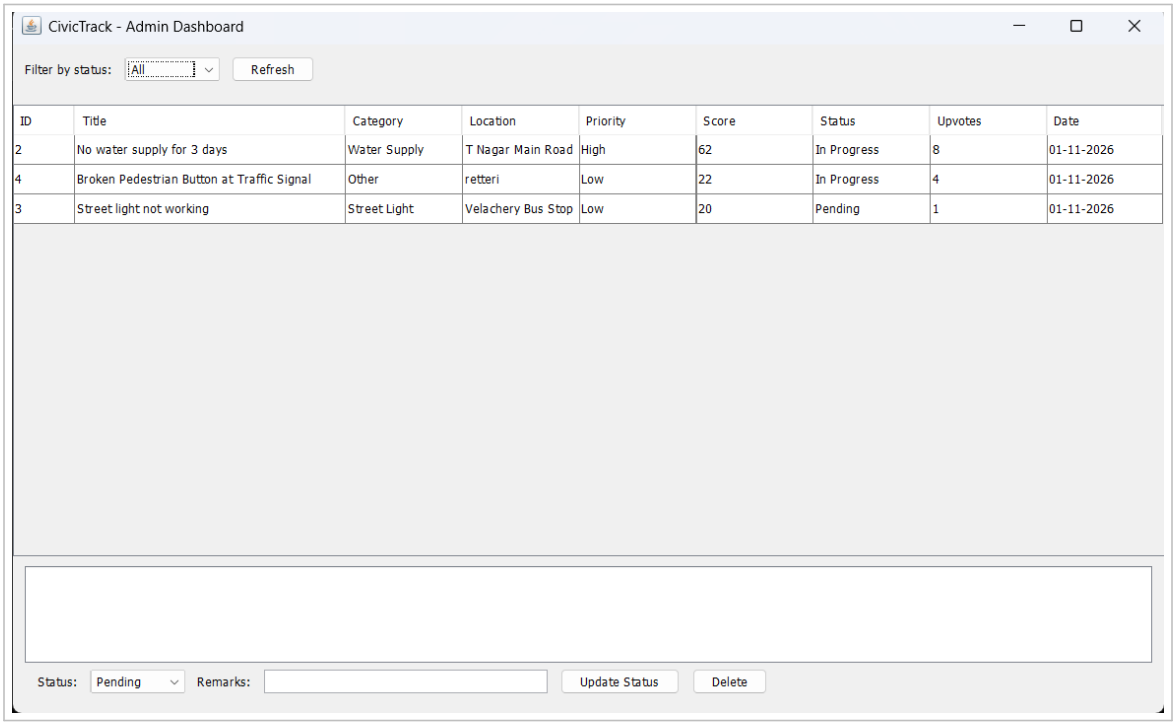
B.3 Complaint Tracking Page



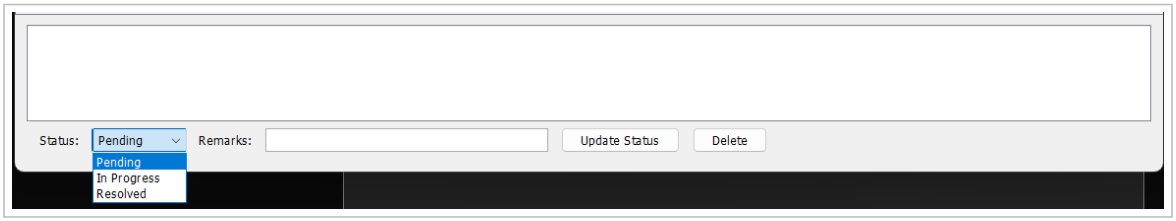
B.4 Admin Login Page



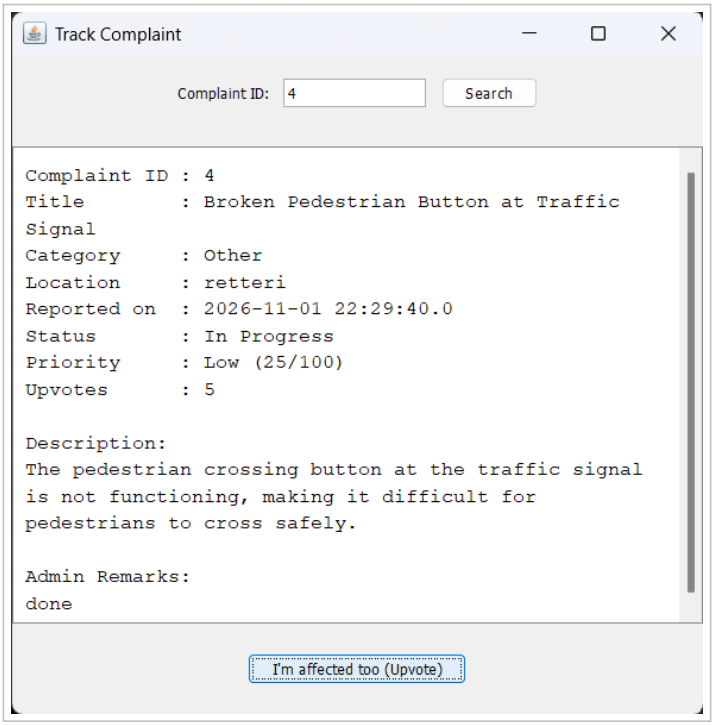
B.5 Admin Dashboard



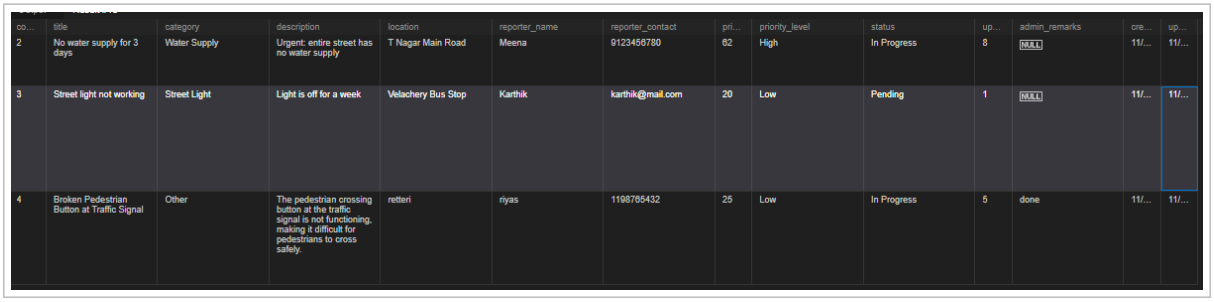
B.6 Complaint Status Update



B.7 Complaint Upvote Functionality



B.8 MySQL Database Records



co...	title	category	description	location	reporter_name	reporter_contact	pri...	priority_level	status	up...	admin_remarks	cre...	up...
2	No water supply for 3 days	Water Supply	Urgent: entire street has no water supply	T Nagar Main Road	Meena	9123456780	62	High	In Progress	8	NULL	11/...	11/...
3	Street light not working	Street Light	Light is off for a week	Velachery Bus Stop	Karthik	karthik@mail.com	20	Low	Pending	1	NULL	11/...	11/...
4	Broken Pedestrian Button at Traffic Signal	Other	The pedestrian crossing button at the traffic signal is not functioning, making it difficult for pedestrians to cross safely.	retteri	riyas	1198765432	25	Low	In Progress	5	done	11/...	11/...

CivicTrack demonstrates the practical application of Java Object-Oriented Programming and MySQL in developing an efficient civic complaint management system. The project provides a structured approach to recording, managing, and monitoring civic issues.

Overall, it fulfils its intended objectives and offers a foundation for future enhancements.